Don't Use a Cannon to Kill a Fly: Lightweight Model Editing for LLMs to Correct Deprecated API Recommendations

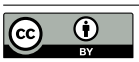
GUANCHENG LIN, City University of Hong Kong, China
XIAO YU*, Zhejiang University, China and Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, China
JACKY KEUNG, City University of Hong Kong, China
XING HU, Zhejiang University, China and Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, China
XIN XIA, Zhejiang University, China and Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, China
ALEX X. LIU, Midea Group, China

Pre-trained or fine-tuned on large code corpora, Large Language Models (LLMs) have demonstrated strong performance in code completion tasks. However, their embedded knowledge is constrained by the timeliness of training data, which often includes code using deprecated APIs. Consequently, LLMs frequently generate deprecated APIs that will no longer be supported in future versions of third-party libraries. While retraining LLMs on updated codebases could refresh their API knowledge, this approach is computationally expensive. Recently, lightweight model editing methods have emerged to efficiently correct specific knowledge in LLMs. However, it remains unclear whether these methods can effectively update deprecated API knowledge and enable edited models to generate up-to-date APIs. To address this gap, we conduct the first systematic study applying 10 state-of-the-art model editing techniques to update deprecated API knowledge in three LLMs: Qwen2.5-Coder, CodeGemma, and DeepSeek-Coder. We introduce `EDAPIBench`, a dedicated benchmark featuring over 70 deprecated APIs from 8 popular Python libraries, with more than 3,000 editing instances. Our results show that the parameter-efficient fine-tuning method AdaLoRA achieves the best performance in enabling edited models to generate correct, up-to-date APIs, but falls short in Specificity (i.e., the editing influences untargeted knowledge). To resolve this, we propose AdaLoRA-L, which defines "Common API Layers" (layers within the LLMs with high importance across all APIs, storing general knowledge and excluded from editing) and restricts edits exclusively to "Specific API Layers" (layers with high importance only for the target API, storing the API-specific knowledge). Experimental results demonstrate that AdaLoRA-L significantly improves Specificity while maintaining comparable performance across other evaluation metrics.

CCS Concepts: • **Software and its engineering → Software libraries and repositories**; **Software maintenance tools**.

*Corresponding Author: Xiao Yu.

Authors' Contact Information: Guancheng Lin, City University of Hong Kong, Department of Computer Science, Hong Kong, China, guanchlin4-c@my.cityu.edu.hk; Xiao Yu, Zhejiang University, The State Key Laboratory of Blockchain and Data Security, Hangzhou, China and Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, Hangzhou, China, xiao.yu@zju.edu.cn; Jacky Keung, City University of Hong Kong, Department of Computer Science, Hong Kong, China, jacky.keung@cityu.edu.hk; Xing Hu, Zhejiang University, The State Key Laboratory of Blockchain and Data Security, Hangzhou, China and Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, Hangzhou, China, xinghu@zju.edu.cn; Xin Xia, Zhejiang University, The State Key Laboratory of Blockchain and Data Security, Hangzhou, China and Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, Hangzhou, China, xin.xia@acm.org; Alex X. Liu, Midea Group, Foshan, China, alexliu@midea.com.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2994-970X/2026/10-ARTISSTA039
https://doi.org/10.1145/3832130

Additional Key Words and Phrases: Large Language Model, Deprecated API, Model Editing

ACM Reference Format:
Guancheng Lin, Xiao Yu, Jacky Keung, Xing Hu, Xin Xia, and Alex X. Liu. 2026. Don't Use a Cannon to Kill a Fly: Lightweight Model Editing for LLMs to Correct Deprecated API Recommendations. *Proc. ACM Softw. Eng.* 3, ISSTA, Article ISSTA039 (October 2026), 23 pages. https://doi.org/10.1145/3832130

1 Introduction

In modern software engineering, rapid iteration and increasing system complexity necessitate the efficient reuse of existing solutions. A fundamental practice is the use of third-party libraries, which provide reusable functionality through Application Programming Interfaces (APIs) [62, 68]. While these libraries accelerate development, they continuously evolve through refactorings [25], bug fixes [19], security patches [59], and feature enhancements, resulting in frequent API updates where older APIs are deprecated and replaced. For example, in PyTorch [4], a popular deep learning library, the API `torch.svd()` has been deprecated in favor of `torch.linalg.svd()` and will be removed in future releases. Consequently, new code should avoid deprecated APIs, as they often lack compatibility with new features or data formats and are eventually removed from the library [1].

This dynamic nature of APIs poses significant challenges for code-assistance tools powered by Large Language Models (LLMs). Although LLMs have demonstrated strong capabilities in code understanding and generation, enabling their widespread use in tasks such as code completion [14, 23, 58], their knowledge is constrained by the timeliness of training data. Many LLMs were trained on codebases using deprecated API versions before up-to-date iterations, leaving them with inherently outdated knowledge. Alternatively, some LLMs may have been trained on mixed datasets containing both deprecated and up-to-date API usages, but their understanding of deprecated APIs may not be fully corrected. Therefore, during code completion, LLMs often suggest deprecated API invocations, leading to suboptimal or erroneous code recommendations. For example, Wang et al. [57] report that 37.4% of GPT-3.5's API predictions are deprecated, highlighting the severity of this issue.

Motivations. To address this problem, Wang et al. [57] proposed two methods, among which `REPLACEAPI` achieved the best performance. `REPLACEAPI` detects deprecated APIs in the LLM's code completion using a maintained deprecated–up-to-date API mapping list, removes the deprecated API and all subsequent tokens, appends the corresponding up-to-date API to form a new prefix, and concatenates it with the original prompt to re-invoke the LLM for code completion. However, this design introduces several practical limitations. In deployment, `REPLACEAPI` must continuously scan all generated completions to check for deprecated APIs, incurring additional detection latency even when no deprecated–up-to-date API replacement is required. As a result, code completions cannot be streamed token-by-token as in standard LLM decoding; instead, the system must wait for the full completion and post-hoc detection before returning the output, leading to a noticeably degraded user experience in interactive settings such as IDEs. Once a deprecated API is detected, `REPLACEAPI` further incurs extra inference cost by re-invoking the LLM with a modified prefix, increasing both latency and token consumption, especially in long-context scenarios. More fundamentally, `REPLACEAPI` performs only surface-level API name substitution without internalizing the semantics of updated APIs, and therefore cannot reliably ensure the correctness of downstream code generation (e.g., valid parameter usage) when API signatures or behaviors change.

A more straightforward solution is to retrain LLMs on the latest codebases that predominantly use up-to-date APIs, thereby overwriting outdated knowledge and enabling the model to generate up-to-date APIs in a single inference pass. However, this approach—akin to "killing a fly with a cannon"—is computationally expensive and time-consuming, rendering it impractical for frequent API updates. Recently, researchers have proposed model editing as a lightweight alternative that avoids full retraining. Model editing aims to update specific pieces of knowledge inside an LLM

efficiently and effectively while leaving unrelated knowledge intact [61]. This technique has been primarily studied in the Natural Language Processing (NLP) domain [10, 15, 27, 36, 37, 52]. For example, in factual knowledge editing, a model might be updated to reflect a change in real-world facts—such as correcting the statement "Joe Biden is the President of USA" to "Donald Trump is the President"—without degrading the model's performance on other unrelated tasks. However, it remains unclear whether existing model editing methods can effectively update deprecated API knowledge in LLMs and enable the edited models to generate up-to-date APIs.

Approaches. To bridge this gap, we conduct the first systematic study on applying 10 state-of-the-art model editing techniques for updating deprecated API knowledge in three LLMs (Qwen2.5-Coder (3B), CodeGemma (2B), and DeepSeek-Coder (1.3B)). To facilitate this, we first propose a dedicated **E**diting **D**eprecated **API Bench**mark, `EDAPIBench`, which can be mostly automatically constructed. We begin with 145 verified API mappings (deprecated → up-to-date) from Wang et al. [57], covering eight popular Python libraries such as PyTorch and TensorFlow. Using these mappings, we extract 65,596 real-world functions from GitHub that call the up-to-date APIs. For each function, we extract lines preceding the API invocation as candidate editing inputs (to prompt LLM API completion) and the invocation line as the target API (ground truth). We then filter these inputs per LLM, retaining only inputs where the original model outputs the deprecated API—ensuring each case genuinely requires editing. This process yields over 900 unique LLM-specific editing instances per model. Next, we extend this core data to build four evaluation subsets that comprehensively assess model editing performance across four key dimensions: *Effectiveness*: Measuring whether the edited model generates the up-to-date API for the original inputs; *Generalization*: Measuring whether it generates the up-to-date API on semantically equivalent but syntactically varied inputs; *Portability*: Measuring whether it generates the up-to-date API across different inputs (with both syntactic and semantic differences) that involve the same deprecated API in their original completions; *Specificity*: Measuring whether it preserves consistent pre-editing behavior on inputs unrelated to the editing task. Among all evaluated model editing methods, the parameter-efficient fine-tuning approach AdaLoRA achieves the highest performance in Effectiveness, Generalization, and Portability. However, it exhibits poor Specificity, primarily because its edits tend to inadvertently modify parameters that encode general knowledge, leading to unintended changes in the model's behavior on inputs unrelated to the editing task.

To address this limitation, we propose an improved variant, AdaLoRA-L, to enhance Specificity while preserving other strengths. Its core design involves: (1) computing gradients of editable parameters for each editing instance to quantify their relevance to the target API; (2) calculating layer importance scores (average squared gradient magnitude of layer parameters) to distinguish "Common API Layers" (high importance across all APIs, storing general knowledge, excluded from editing) and "Specific API Layers" (high importance only for the target API, storing API-specific knowledge, set as editing targets); (3) restricting edits exclusively to Specific API Layers to avoid interfering with general knowledge. Results show AdaLoRA-L significantly improves Specificity by 79.1%, 67.6%, 45.5% across the three target edited models, while maintaining AdaLoRA's strong performance in Effectiveness, Generalization, and Portability. In addition, AdaLoRA-L outperforms `REPLACEAPI` in Effectiveness, Generalization, and parameter-correct API generation by updating the model's internal knowledge, while `REPLACEAPI` has higher inference and token costs and is more suitable when API name correctness matters more than parameter accuracy or user experience. In summary, our paper makes the following contributions:

(1) **Benchmark:** We construct `EDAPIBench`—the first dedicated benchmark for evaluating deprecated API knowledge editing in LLMs, with mostly automated construction, serving as a standardized, rigorous platform that benefits both software engineering and broader NLP communities.

(2) **Evaluation:** We evaluate 10 state-of-the-art model editing techniques on three LLMs, and find AdaLoRA excels in three key dimensions but lacks Specificity due to inadvertently modifying parameters encoding general knowledge.

(3) **Method:** We propose AdaLoRA-L, an improved model editing approach that isolates API-specific layers via gradient-based importance scoring, addressing AdaLoRA's Specificity limitation while retaining its strengths in Effectiveness, Generalization, and Portability.

2 Task Definition

Our study frames the model editing scenario as a code completion task, with the primary goal of updating deprecated API knowledge within LLMs. In this context, we denote the pre-editing LLM as f and define an editing instance as a tuple $M = (x, y)$, where x represents the editing input (a code snippet to be completed) and y is the editing target (the specific correct, up-to-date API that should replace the deprecated one). For a given input x, the current API completion result produced by the original LLM f is denoted as y_d, which corresponds to the deprecated API call. The goal of model editing is to enable the edited LLM (denoted f_e) to generate the desired target y for the input x, effectively correcting the deprecated API call y_d. Following existing works [20, 28, 61], we define that an effective model edit should meet the following four key criteria.

Effectiveness ensures that the edited LLM f_e correctly generates the up-to-date API y for the editing input x: $f_e(x) = y$.

Generalization requires that for any input x_g which is a syntactic variation of the original input x but semantically equivalent (i.e., both x and x_g have the same meaning or behavior), and for which the original LLM f still completes with the deprecated API y_d for x_g, the edited LLM f_e should generate the correct, up-to-date API y for x_g: $f_e\left(x_g\right) = y$.

Portability requires that for any input x_p, which represents a real-world input that is both semantically and syntactically different from x and is completed by the original LLM f using the deprecated API y_d, the edited LLM f_e should generate the up-to-date API y for x_p: $f_e\left(x_p\right) = y$.

Specificity ensures that model editing on the original LLM f does not affect other irrelevant knowledge, preserving unchanged outputs for inputs irrelevant to the editing task. Let $\mathbb{U} = \{U_1, U_2, \ldots\}$ denote a set of unrelated instances, where each $U = (x_u, y_u)$ consists of a non-target input x_u, and its corresponding API completion output by the original LLM f (i.e., $y_u = f(x_u)$). Here, x_u is unrelated to the editing input x and involves no overlapping API calls with the target API y or the deprecated API y_d. The edited LLM f_e should satisfy: $f_e\left(x_u\right) = f\left(x_u\right), \forall U \in \mathbb{U}$.

3 Study Design

3.1 Model Editing Techniques and Subjects

The model editing methods can be categorized into four types based on the way knowledge is updated [20, 61]. **Locate-then-edit**: This paradigm first locates where knowledge is stored and then modifies the corresponding parameters. **Parameter-efficient Fine-tuning**: Unlike full fine-tuning, this method focuses on fine-tuning only the parameters of specific components of the model, thereby reducing computational and memory overhead. While such methods are not specifically designed for knowledge editing, several studies [20, 28, 73] adopt them as baseline methods, as this efficient model updating approach aligns with the requirements of model editing. **Memory-based method**: Instead of modifying model parameters, this approach introduces an additional memory module to store new knowledge, which is queried when relevant inputs are encountered. **Meta Learning**: This method trains a hypernetwork to predict how parameters should shift when editing a given instance. During the editing phase, for each edited instance, the hypernetwork is utilized to calculate the parameter shift, and the predicted shifts are subsequently applied to the model's

Table 1. The brief descriptions of the ten model editing methods.

Locate-then-edit	
ROME [36]	It utilizes causal tracing to identify a particular middle-layer Feed Forward Network (FFN) where knowledge is stored. ROME conceptualizes model editing as a constrained least squares problem, implementing a rank-one update to the FFN's projection weights to incorporate new knowledge.
MEMIT [37]	This is an enhanced version of ROME, allowing for multi-layer editing. It modifies key FFNs crucial for knowledge recall by distributing intended changes across these layers as key-value memories.
PMET [27]	It analyzes the hidden states of Multi-Head Self-Attention (MHSA) and FFN, showing that MHSA acts as a knowledge extractor encoding general patterns without needing weight updates. Thus, PMET focuses on updating FFN weights for knowledge updates.
AlphaEdit [10]	It projects parameter perturbations onto the key matrix's null space to safeguard knowledge, preserving existing information undisturbed. By prioritizing minimal update errors, it maintains hidden representation stability, preventing forgetting during editing.
Parameter-efficient Fine-tuning	
FT-L [77]	It directly fine-tunes the FFN of a single layer and minimizes the impact on irrelevant data by constraining parameter norms.
LoRA [18]	It fine-tunes the attention modules across all layers. It first freezes the model weights and introduces low-rank matrices into the attention modules, minimizing the volume of parameter updates by optimizing only these matrices.
AdaLoRA [74]	As an optimized version of LoRA, it also fine-tunes the attention modules across all layers. The difference is that it dynamically allocates more parameter update budget to critical attention layers, while assigning fewer updates to less important ones.
Memory-based	
GRACE [15]	It introduces a module with a discrete key-value codebook for storing edited knowledge and a deferral mechanism. This mechanism enables the codebook to identify modified key-value pairs, whose values then replace the hidden layer output of the original model.
A-GRACE [28]	This is an improved version of GRACE. By incorporating a contrastively-trained encoder, it successfully boosts GRACE's generalization without significantly impacting other performance dimensions.
Meta Learning	
MALMEN [52]	Instead of computing gradients to fine-tune model parameters, it uses a hypernetwork to calculate parameter shifts for editing instances, applying the shifts to the model.

parameters to perform the edit. We investigate 10 state-of-the-art model editing methods across these four categories, with details summarized in Table 1. The selected methods include all those studied by Li et al. [28], a recent empirical study on model editing in software engineering, and additionally include AlphaEdit, LoRA, and AdaLoRA to enhance generalizability.

We select the three popular open-source code LLMs as our editing subjects, which are widely adopted in various software engineering tasks [24, 41, 50, 51]. **Qwen2.5-Coder (3B)** [21]: Proposed by Alibaba and released in September 2024, it is trained on 5.5 trillion tokens collected before February 2024. **CodeGemma (2B)** [53]: Developed by Google and released in April 2024, it is trained on 1 trillion tokens from publicly available code repositories dating up to early 2024. **DeepSeek-Coder (1.3B)** [13]: Developed by DeepSeek and released in June 2024, it is trained from scratch on 2 trillion tokens collected before February 2023.

3.2 EDAPIBench Construction

We construct `EDAPIBench`, which comprises over 900 editing instances $M = (x, y)$ per LLM. For each instance, the editing input x is a code completion prompt—specifically, a prompt that should

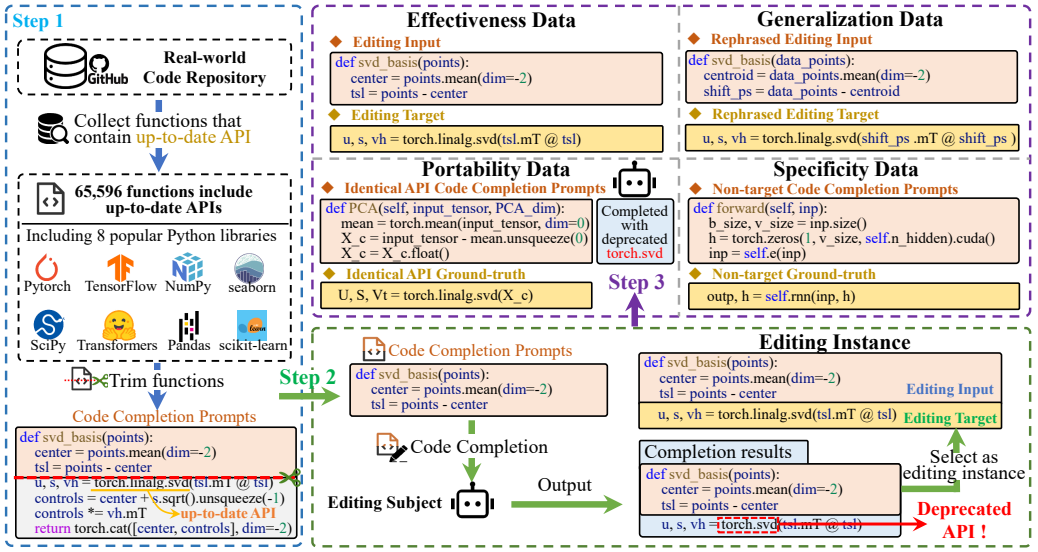


Fig. 1. The construction process of EDAPIBench.

yield the target up-to-date API y, but which the original LLM f completes with the deprecated API y_d. The editing target is the corresponding up-to-date API y. Each editing instance serves as core data to evaluate the Effectiveness of model editing; additionally, for every instance, we construct Generalization, Portability, and Specificity datasets to assess the performance of the edited LLM f_e across the three key dimensions. Figure 1 illustrates the construction process of EDAPIBench.

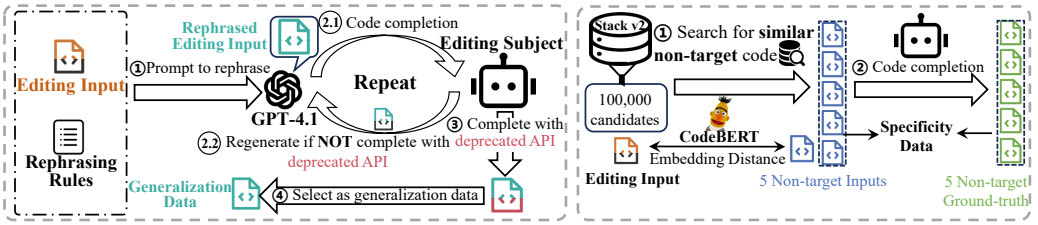
Step 1: Code Completion Prompts Collection. This step focuses on collecting code snippets that can serve as editing inputs x—specifically, prompts that require the up-to-date API y for correct completion. Using the 145 API mappings from Wang et al. [57], which pair deprecated APIs y_d with up-to-date replacements y across eight popular Python libraries (Figure 3), we search open-source repositories via the Sourcegraph tool [5], yielding 65,596 functions with calls to target up-to-date APIs. For each function, the lines preceding the API invocation serve as candidate inputs x, and the invocation line itself as the ground truth for the target API y.

Step 2: Data Filtering. We ensure that the three LLMs introduced in Section 3.1 produce the deprecated API y_d for the candidate prompts before model editing. To verify this, we analyze completions by inputting each candidate editing input into each LLM separately and prompting each to complete the input three times with temperature set to 0 (using greedy sampling). We retain only those prompts where the model outputs the deprecated API in all three completions. This yields valid, LLM-specific editing instances tailored to each model that truly require model editing. For instance, as shown in Figure 1, the editing input `def svd_basis(points): ...tsl=points-center` is retained since the LLM generates the deprecated API `torch.svd()` in all completions.

Step 3: Evaluation Data Preparation.

1) Effectiveness Data. The editing instances $M = (x, y)$ themselves constitute the Effectiveness data. After model editing, Effectiveness is evaluated by checking whether the edited LLM f_e can correctly complete the prompt x with the target API y.

2) Generalization Data. The generalization dimension evaluates whether the edited model f_e can successfully complete the up-to-date API call y based on code completion prompts that are semantically identical but syntactically different from the original editing input x. This requires rephrasing the code of the original editing input while preserving their meanings and behaviors.



(a) Generalization Data (b) Specificity Data

Fig. 2. The construction process of generalization data and specificity data in EDAPIBench.

Inspired by Yu et al. [66], who proposed 18 rephrasing rules capable of altering code while preserving semantic and syntactic naturalness, we adopt 14 of these rules, excluding four that are inapplicable to Python language (e.g., converting "switch-case" to "if-else"). As illustrated in Figure 2, and following Wang et al. [57], we provide these 14 rephrasing rules to GPT-4.1 [2], instructing it to apply them as extensively as possible to generate rephrased editing inputs that are maximally dissimilar from the original. We then prompt the editing subjects (i.e., the LLMs) to complete these rephrased inputs. If the completion does not contain the deprecated API y_d, GPT-4.1 is instructed to further rephrase the input, with a maximum of ten attempts. Therefore, a small number of samples still fail to elicit completions containing the deprecated API after repeated rephrasing. For all rephrased samples, the first two authors independently review them to confirm that the rephrasings are reasonable and do not alter the semantics of the context preceding the API. These validated rephrased inputs are then used as generalization data. For example, as shown in Figure 1, the rephrased input appears as `def svd_basis(data_points): ... shift_ps=data_points-centroid`.

3) Portability Data. The portability dimension evaluates whether the edited model can correctly complete the up-to-date API call on different editing instances where the original model also completes the prompt with the same deprecated API. For each editing instance $M = (x, y)$, we randomly select another editing instances with the same target API y from EDAPIBench as portability evaluation data. For example, as shown in Figure 1, one such selected editing instance is: x= "`def PCA (self, input_tensor, PCA_dim): X_c=X_c.float()` ", y="`torch.linalg.svd()`".

4) Specificity Data. The Specificity dimension requires the edited LLM f_e to preserve consistent outputs for non-target inputs—defined as inputs for which the original LLM f does not generate either the target API y or the deprecated API y_d during completion. In line with Wang et al. [57], our benchmark focuses on the most challenging scenario: whether f_e exhibits output shifts for non-target inputs that are semantically similar to the original editing input x. As shown in Figure 2, we randomly select 100,000 Python files from The Stack v2 dataset [33] as non-target input candidates. Since The Stack v2 has an extremely large scale, full retrieval would be computationally expensive, so this sampling approach is adopted for practicality. To collect semantically similar non-target inputs, we employ CodeBERT [11]—a widely used code embedding model in software engineering—to generate embeddings for each editing input x. For every x, we select five non-target inputs with the smallest embedding distance from the candidates, while ensuring that these inputs contain neither the target API y nor the deprecated API y_d from the corresponding editing instance. We then instruct the original LLMs (editing subjects) to complete these five non-target inputs prior to model editing, and preserve their API outputs as "non-target ground truth". These non-target inputs and their corresponding ground truth together constitute the Specificity data. For example, as shown in Figure 1, the non-target API for one piece of Specificity data is `self.nn()`.

Dataset Statistics. Table 2 lists the editing instances and APIs in EDAPIBench. After excluding APIs that LLMs cannot reliably generate in deprecated form, 82 unique deprecated–up-to-date API pairs remain from the original 145 mappings provided by Wang et al. [57]. Generalization data

Table 2. The number of instances and APIs for different LLMs across evaluation dimensions in EDAPIBench.

Models	Qwen2.5-Coder		CodeGemma		DeepSeek-Coder	
Statistical value	# instances	# APIs	# instances	# APIs	# instances	# APIs
Effectiveness	1,401	75	905	71	1,072	76
Generalization	1,401	75	905	71	1,072	76
Portability	1,390	67	892	58	1,064	65
Specificity	7,005	3,375	4,525	2,167	5,360	2,626

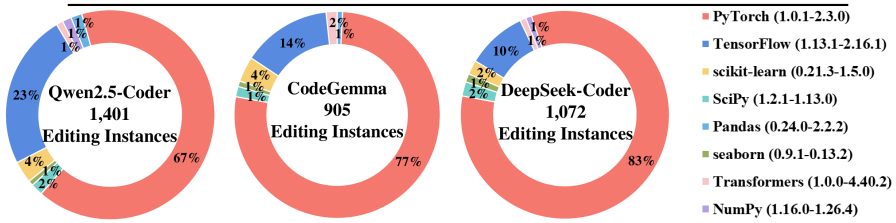


Fig. 3. The distribution of the number of target APIs from different libraries in EDAPIBench.

matches Effectiveness in volume, as each instance has a rephrased counterpart. Specificity data is five times larger, with five non-target inputs per instance. Portability data is slightly smaller, since some APIs have only one editing instance—preventing the selection of another instance with the same deprecated API. Figure 3 illustrates the distribution of editing instances across libraries.

3.3 Experiment Settings

All experiments are conducted on RTX 4090 GPUs paired with an Intel Core i9 processor. Following existing code completion works [67, 69], we use incomplete code snippets directly as prompts for code completion, without adding any additional instructions. For the method requiring training (i.e., MALMEN), we follow existing works [9, 28, 76], splitting EDAPIBench into training and test sets at a 1:1 ratio using a two-fold cross-validation approach, and the results of the two rounds of testing are integrated to serve as the final results. Regarding the selection of editing layers for locate-then-edit methods, we follow Li et al. [28] and maintain consistency with prior benchmarks [60, 61]. Since these benchmarks have already identified the key editing layers via causal intervention [36], we directly edit the same layers and therefore do not perform additional causal intervention in our experiments. For the hyperparameters of model editing methods, we keep them consistent with the settings used in existing model editing benchmarks [28, 60, 61]. Our preliminary experiments indicate that varying the hyperparameters and editing layers for locate-then-edit methods does not lead to significant performance changes, and the overall conclusions remain unchanged, with AdaLoRA consistently outperforming other baselines across the first three dimensions.

ROME, MEMIT, PMET, AlphaEdit: In the case of ROME, we set the 5th layer as the editing layer, while for the other three methods, we choose the {4, 5, 6, 7, 8}-th layers as the editing layers. All methods are trained with 25 iterations and a learning rate of 5e-1.

GRACE, A-GRACE: We insert an adapter into the down projection matrix of an MLP from the later layers. The initial deferral radius of the key is set to 1, and the value corresponding to this key is fine-tuned for 30 iterations with a learning rate of 1. For the additional encoder introduced in A-GRACE, we follow the settings in the original study [28], with a hidden layer dimension of 256, and train it for 100 epochs at a learning rate of 1e-4.

MALMEN: The projection matrices of the last 5 layers of LLMs serve as the update targets for MALMEN. Initially, a hypernetwork containing two MLPs with a hidden layer dimension of 1920 is trained on the training set for predicting parameter shifts. This training process involves 1,000 iterations and a learning rate of 1e-5. During editing, the hidden states of the editing instance at

the editing layers are cached first. Subsequently, the hypernetwork predicts the parameter shifts to facilitate the weight update process.

FT-L, LoRA, AdaLoRA: We set an MLP from the later layers as the editing layer for FT-L to conduct fine-tuning, with 40 epochs and a learning rate of 5e-4. For LoRA and AdaLoRA, we set 30 training epochs, a rank of 8, and a learning rate of 5e-3.

The performance of model editing is evaluated based on how closely the single-line completion outputs of edited LLMs match the ground-truth. Following Li et al. [28], we utilize three metrics: **Exact Match** (EM), **BLEU** [42], and **ROUGE-L** [29]. Additionally, we introduce **API Exact Match** (AEM), specifically tailored for the EDAPIBench scenario. EM measures whether the LLM's single-line completion exactly matches the entire ground-truth line at the token level, assigning a score of 1 for a perfect match and 0 otherwise. AEM is a variant of EM that only checks whether the single-line completion contains the ground-truth API, regardless of other tokens. BLEU [42] and ROUGE-L [29] are two commonly adopted metrics in code-related tasks [6, 34, 55, 70, 71]. To reduce randomness, all methods are run five times, and the median is reported as the final performance on EDAPIBench.

4 Experimental Results

4.1 RQ1: How do the methods perform at editing deprecated API knowledge in LLMs?

As shown in Tables 3, 4, and 5, no single method consistently excels across all four dimensions: Effectiveness, Generalization, Portability, and Specificity. Take AdaLoRA as an example—a standout method that achieves the best overall performance in three dimensions (Effectiveness, Generalization, and Portability). However, its Specificity is noticeably lower than that of methods like GRACE. Conversely, while GRACE excels in Specificity, it exhibits a clear performance gap compared to AdaLoRA in Generalization and Portability. Although A-GRACE improves upon GRACE's Generalization, the gap between A-GRACE and AdaLoRA in Portability remains considerable, with a difference of 0.149 to 0.724 in AEM. This limitation likely stems from its contrastive training process: the model primarily optimizes for generalization data involving simple rephrasing, where the core context of the input remains largely consistent with the original editing instance. Consequently, it only learns to adapt to such straightforward rephrasing patterns. When confronted with portability data—inputs that differ substantially from the original editing input in both semantics and syntax, A-GRACE struggles to effectively apply the edited API knowledge to these more complex, scenario-specific cases. Additionally, compared to AdaLoRA, LoRA was originally designed as a general-purpose parameter-efficient fine-tuning method suitable for tasks such as text classification and general generation. Its goal is low-cost adaptation rather than precise knowledge updating. Regardless of the importance of parameters to the target API knowledge, LoRA adjusts all parameters using low-rank matrices of the same dimension, which leads to suboptimal overall performance in this context. Although locate-then-edit methods have demonstrated strong performance in natural language processing tasks, they perform poorly on our benchmark. These methods are designed to edit factual knowledge in natural language, which can often be represented as explicit "subject–relation–object" triples (e.g., <US President, is, Trump>). Accordingly, they treat MLP layers in LLMs as key–value memories, where the subject and relation serve as the key and the object as the value. Following Li et al. [28], we adopt API-preceding code context as the key and the target API line as the value in the code domain. However, unlike the clear semantic mappings in natural language, the relationship between diverse code contexts and APIs is far more implicit. This lack of well-defined semantic triples makes locate-then-edit methods inherently ill-suited for deprecated API updates. Meta-learning methods, on the other hand, show no clear advantages or obvious shortcomings across the four dimensions, resulting in mediocre overall performance.

Table 3. The performance of the methods on editing deprecated API knowledge within Qwen2.5-Coder.

Editor	Effectiveness				Generalization				Portability				Specificity			
	AEM	EM	BU	RL	AEM	EM	BU	RL	AEM	EM	BU	RL	AEM	EM	BU	RL
Pre-editd	2.7	0.7	40.4	63.4	5.1	1.0	36.9	58.2	2.6	0.7	39.8	62.9	99.3	93.7	97.6	97.2
ROME	2.0	0.4	6.6	17.5	2.9	0.9	7.9	18.8	4.4	1.8	15.0	29.1	43.8	24.7	40.9	52.2
MEMIT	6.5	2.3	26.4	46.0	6.7	2.4	26.1	44.4	4.8	1.3	36.3	58.0	91.4	81.6	89.1	91.9
PMET	5.9	2.2	35.0	56.3	7.2	2.1	33.1	53.0	3.4	0.9	38.6	61.4	96.2	88.6	94.4	95.1
AlphaEdit	7.1	2.9	15.8	31.9	6.7	2.6	17.8	33.5	5.8	1.5	36.2	58.3	85.2	72.6	82.6	87.2
GRACE	97.8	55.5	73.0	80.9	6.3	1.4	37.3	58.4	2.8	1.0	39.9	62.9	**99.2**	**93.4**	**97.5**	**97.2**
A-GRACE	96.2	59.7	75.6	83.4	88.3	37.3	65.2	74.1	47.6	10.5	30.0	38.2	97.8	89.6	97.7	97.6
MALMEN	53.8	15.1	46.0	60.7	48.6	10.1	40.5	54.1	42.9	9.4	41.9	57.7	61.2	41.2	57.9	68.4
FT-L	58.4	33.8	67.4	78.7	48.8	22.3	56.2	69.7	16.2	5.9	44.8	**65.6**	96.9	90.1	95.0	95.5
LoRA	24.0	0.0	9.7	26.1	17.4	0.0	6.6	20.2	7.8	0.0	2.3	10.6	5.5	1.2	0.1	1.3
AdaLoRA	**98.1**	**60.4**	**81.0**	**87.6**	**89.5**	**39.3**	**69.9**	**77.7**	**71.4**	**11.7**	**49.7**	64.2	49.2	28.4	47.9	58.3

Table 4. The performance of the methods on editing deprecated API knowledge within CodeGemma.

Editor	Effectiveness				Generalization				Portability				Specificity			
	AEM	EM	BU	RL	AEM	EM	BU	RL	AEM	EM	BU	RL	AEM	EM	BU	RL
Pre-editd	1.0	0.4	37.6	60.8	4.5	1.6	29.4	49.4	0.8	0.3	36.2	59.7	99.5	98.3	99.2	96.8
ROME	1.2	0.0	1.9	12.4	0.9	0.1	1.8	9.8	6.6	1.1	17.8	34.0	42.8	28.5	38.4	49.3
MEMIT	3.5	0.1	4.5	17.5	4.2	0.7	3.6	14.5	5.3	1.0	31.0	52.2	89.8	85.4	88.8	90.2
PMET	9.4	1.6	16.1	33.7	8.5	1.7	14.3	29.0	3.7	0.9	34.5	57.3	95.2	91.7	94.2	93.8
AlphaEdit	4.2	0.1	4.7	18.0	3.2	0.4	4.0	14.5	5.4	1.2	32.7	54.1	88.4	83.6	87.1	89.1
GRACE	24.2	12.3	46.7	65.3	4.6	1.6	29.3	49.4	1.0	0.3	36.7	60.1	97.3	**94.7**	**96.8**	95.3
A-GRACE	24.1	11.9	48.7	66.3	18.1	4.9	33.8	52.1	1.8	0.4	36.3	59.2	**97.5**	94.2	96.6	**95.4**
MALMEN	34.7	8.2	32.3	44.7	25.2	4.0	22.8	34.7	37.6	7.7	38.3	52.4	49.3	32.4	45.2	57.2
FT-L	89.0	91.3	95.0	96.3	69.4	33.7	59.8	67.7	52.5	13.2	**52.0**	**68.0**	88.7	81.4	87.2	89.2
LoRA	3.2	0.0	2.4	12.6	2.9	0.0	1.5	9.4	2.2	0.0	1.1	7.3	8.5	1.1	0.1	0.5
AdaLoRA	**99.0**	**91.7**	**95.3**	**96.7**	**78.7**	**39.4**	**65.1**	**71.3**	**74.2**	**13.6**	49.2	61.9	51.2	34.0	46.5	56.4

Table 5. The performance of the methods on editing deprecated API knowledge within DeepSeek-Coder.

Editor	Effectiveness				Generalization				Portability				Specificity			
	AEM	EM	BU	RL	AEM	EM	BU	RL	AEM	EM	BU	RL	AEM	EM	BU	RL
Pre-editd	3.3	0.8	37.2	59.2	4.3	0.8	35.3	55.5	2.2	0.4	37.7	60.0	86.4	78.7	85.8	88.5
ROME	0.3	0.0	0.4	3.4	0.2	0.0	0.3	3.0	1.3	0.3	5.0	14.5	32.4	19.9	28.5	38.9
MEMIT	4.8	0.7	21.5	42.3	4.3	0.6	20.4	39.3	3.7	0.4	35.8	57.7	83.9	75.4	83.4	86.9
PMET	5.5	0.8	32.6	54.4	5.9	1.2	31.7	51.5	2.6	0.5	37.5	59.5	85.4	77.6	84.7	87.9
AlphaEdit	0.8	0.1	1.1	7.7	0.4	0.0	1.0	7.1	4.0	0.4	27.8	47.7	67.2	56.3	65.8	73.2
GRACE	71.6	39.8	73.0	83.9	6.9	1.9	36.7	56.8	2.2	0.4	37.8	60.1	**86.3**	**78.6**	**85.8**	**88.4**
A-GRACE	73.3	40.4	72.8	83.9	63.2	21.3	57.7	70.0	43.1	4.9	40.7	56.8	83.1	75.2	85.5	88.2
MALMEN	38.6	4.8	35.0	49.9	37.8	4.7	32.0	45.8	38.6	4.5	37.2	53.1	56.4	39.5	51.1	61.9
FT-L	57.2	12.7	54.0	69.1	52.4	9.1	48.3	63.3	40.8	6.7	**47.8**	**65.2**	86.1	78.3	85.5	88.3
LoRA	7.7	0.9	6.5	12.1	7.7	0.4	5.0	9.8	4.2	0.0	1.5	4.2	8.5	1.4	0.1	0.7
AdaLoRA	**98.3**	**89.5**	**94.7**	**96.5**	**93.9**	**56.9**	**84.2**	**88.1**	**58.0**	**7.8**	45.5	61.5	55.0	37.5	54.6	64.3

Answer to RQ1: No single method performs well across all four dimensions. AdaLoRA performs best in Effectiveness, Generalization, and Portability, but underperforms in Specificity.

4.2 RQ2: What is the efficiency of the methods for deprecated API knowledge editing?

Figure 4 illustrates the average time cost per editing instance for each model editing method. Methods in the locate-then-edit category incur the highest time costs: PMET, the most costly, averages 21.2 seconds per edit across models, while ROME—more efficient than peers in this category—still averages 7.6 seconds per edit. Among all methods, MALMEN demonstrates the lowest time cost, with an average editing time of less than 1 second per edit. Memory-based methods and parameter-efficient fine-tuning approaches show similar time costs, ranging from 2.0 to 5.2

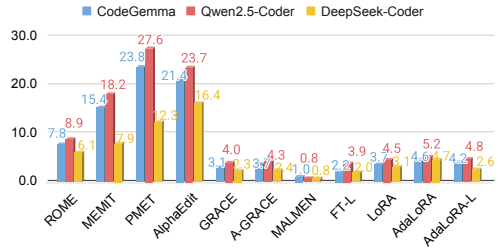


Fig. 4. The average time cost (seconds) of the model editing methods per edit.

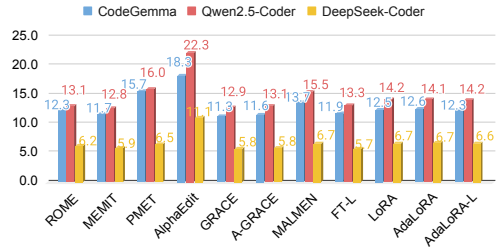


Fig. 5. The average peak memory cost (GB) of the model editing methods per edit.

seconds per edit. AdaLoRA-L (averaging 3.8 seconds per edit) is faster than AdaLoRA (averaging 4.8 seconds per edit), primarily because it edits fewer parameters, resulting in reduced computational load. The layer identification step in AdaLoRA-L incurs a one-time offline cost of 52s, 117s, and 189s across models. Since this step is performed only once before editing and reused for all subsequent edits, its amortized overhead is negligible at approximately 0.1s per edit. Figure 5 presents the average peak memory cost per editing instance for each method. PMET and AlphaEdit have the highest memory cost (12.7GB and 17.2GB on average across models), while MEMIT is the most memory-efficient at 10.1GB. The remaining methods have similar memory cost, averaging around 11.8 GB across the three models.

> **Answer to RQ2:** The model editing methods generally have low time and memory overhead, allowing a single edit on a 3B-parameter model to complete within a few seconds on a 24 GB GPU.

4.3 RQ3: How to improve the performance of AdaLoRA?

4.3.1 Approach. To address RQ3, we aim to improve AdaLoRA, which achieves the highest Effectiveness, Generalization, and Portability among the methods explored in RQ1. Despite these strengths, AdaLoRA performs poorly in Specificity, likely because indiscriminate editing across all layers modifies parameters storing general knowledge, thereby disrupting information unrelated to the target API. A natural approach to enhance Specificity is to reduce the number of layers edited by AdaLoRA, thereby limiting the scale of parameter updates and avoiding unnecessary changes to general knowledge. However, selecting which layers to edit presents a critical challenge: failing to select layers relevant to the target API knowledge would degrade Effectiveness, Generalization, and Portability, while selecting layers that store general knowledge for editing would still compromise Specificity. To overcome this challenge, we propose AdaLoRA-L, an improved variant of AdaLoRA. It first precisely identifies layers storing general knowledge, then identifies API-specific layers for each target API from the remaining layers, and restricts editing operations exclusively to these API-specific layers. This design ensures that Specificity is enhanced without sacrificing performance in other dimensions. The overall layer selection process for AdaLoRA-L is illustrated in Figure 6.

Gradient Computation: For each editing instance, we have the LLM perform code completion on the editing input. We then calculate the loss based on the output—specifically, we calculate the loss on the target API token (i.e., the tokens corresponding to the up-to-date API in the editing target) while ignoring loss from other tokens. Next, we backpropagate this loss to calculate the gradient for each editable parameter across all layers. Consistent with neural network pruning methods [40, 75], which employ gradients as a metric to quantify parameter importance with respect to specific samples, these computed gradients directly indicate a parameter's relevance to the target API: a larger absolute gradient value signifies that even minor adjustments to the

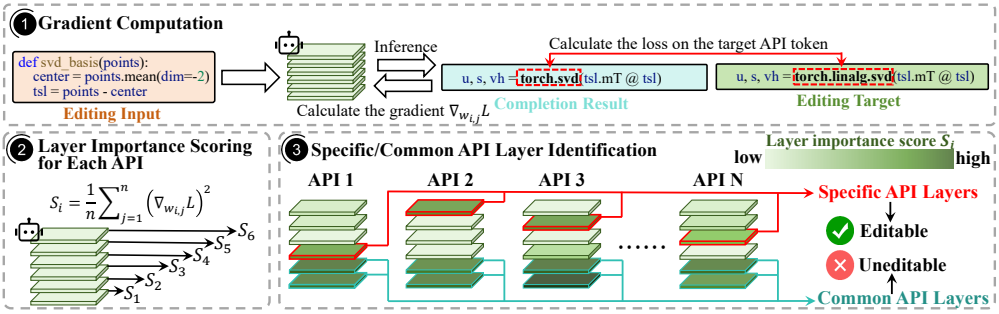


Fig. 6. The identification process of Specific API Layers and Common API Layers.

parameter would substantially impact the model's capacity to output the correct up-to-date API token, meaning such parameters are critical for updating the current API.

Layer Importance Scoring for Each API: Using these parameter gradients, we further compute the layer importance score with the following formula: $S_i = \frac{1}{n}\sum_{j=1}^{n}\left(\nabla_{w_{i,j}} L\right)^2$, where S_i denotes the importance score of the i-th layer, L is the loss computed on the up-to-date API tokens, w_{ij} represents the j-th editable parameter in the i-th layer, and n is the total number of editable parameters in that layer. Intuitively, this score quantifies the average squared gradient magnitude of all editable parameters in the layer, reflecting the layer's overall importance for enabling the model to predict the up-to-date API token.

For each editing instance, we calculate the importance scores of all layers. Then, for each API, we average the scores across all its editing instances to obtain API-specific layer importance scores, which indicate each layer's relevance to the API. By analyzing the scores across all APIs, we identify:

Common API Layers: Layers that exhibit high importance across all APIs. These layers encode general knowledge critical to all APIs and are excluded from editing;

Specific API Layers: Excluding Common API Layers, these are layers that demonstrate high importance only for the target API. These layers store the target API-specific knowledge and serve as the core targets for editing.

Through this layer identification and selection process, AdaLoRA-L precisely isolates a set of specific API layers for each target API. These layers constitute the focused editing scope, ensuring parameter updates concentrate on API-specific knowledge while avoiding interference with general knowledge layers. This design enhances the Specificity of model editing and maximally preserves performance in Effectiveness, Generalization, and Portability.

4.3.2 Results. We set hyperparameters for AdaLoRA-L on the three models as follows: the number of frozen Common API Layers is set to 8, 8, and 4 for Qwen2.5-Coder, DeepSeek-Coder, and CodeGemma, respectively; the number of edited Specific API Layers is set to 4 for CodeGemma and 8 for the other two models. Table 6 presents the performance improvements of AdaLoRA-L compared to AdaLoRA. To evaluate the statistical significance of differences between AdaLoRA-L and baselines, we employ the Wilcoxon signed-rank test [63] with the Benjamini-Hochberg correction procedure [54]. An asterisk (*) following the performance value indicates a statistically significant difference. In terms of Effectiveness, AdaLoRA-L achieves consistent performance gains across all three models. In terms of Generalization, it outperforms AdaLoRA on two models, with only a minor deficit observed on DeepSeek-Coder. Notably, in both Portability and Specificity, AdaLoRA-L consistently surpasses AdaLoRA across all three LLMs. Specifically, Portability shows relative improvements of 4.1%, 13.6%, and 29.8% on the three models in terms of AEM. Specificity exhibits substantial relative gains of 79.1%, 67.6%, and 45.5% in AEM. Moreover, statistical tests

Table 6. The performance comparison of of AdaLoRA-L, AdaLoRA, and REPLACEAPI.

Editor	Effectiveness				Generalization				Portability				Specificity			
	AEM	EM	BU	RL	AEM	EM	BU	RL	AEM	EM	BU	RL	AEM	EM	BU	RL
Qwen2.5-Coder																
Pre-editd	2.7	0.7	40.4	63.4	5.1	1.0	36.9	58.2	2.6	0.7	39.8	62.9	99.3	93.7	97.6	97.2
AdaLoRA	98.1	60.4	81.0	87.6	89.5	39.3	69.9	77.7	71.4	11.7	49.7*	64.2*	49.2*	28.4*	47.9*	58.3*
REPLACEAPI	97.8	22.8*	67.4*	79.2*	84.4*	17.9*	58.1*	70.3*	**97.2***	**22.8**	**67.4***	**78.6***	**99.2***	**93.5***	**97.4***	**97.1***
AdaLoRA-L	**98.2**	**63.9**	**84.6**	**89.7**	**90.8**	**42.4**	**74.9**	**81.2**	74.3	21.4	60.6	70.3	88.1	76.9	85.0	88.8
CodeGemma																
Pre-editd	1.0	0.4	37.6	60.8	4.5	1.6	29.4	49.4	0.8	0.3	36.2	59.7	99.5	98.3	99.2	96.8
AdaLoRA	99.0	91.7	95.3	96.7	78.7	39.4	65.1	71.3	74.2*	13.6	49.2	61.9*	51.2*	34.0*	46.5*	56.4*
REPLACEAPI	98.2	21.7*	64.5*	75.9*	66.2*	12.0*	44.9*	57.4*	**96.6***	**22.7**	**63.3***	**74.9**	**98.0***	**96.3***	**97.5***	**96.0***
AdaLoRA-L	**99.2**	**91.9**	**95.9**	**97.2**	**81.8**	**41.5**	**67.8**	**74.5**	84.3	17.1	58.0	71.0	85.8	78.8	85.3	87.8
DeepSeek-Coder																
Pre-editd	3.3	0.8	37.2	59.2	4.3	0.8	35.3	55.5	2.2	0.4	37.7	60.0	86.4	78.7	85.8	88.5
AdaLoRA	98.3	89.5	94.7	96.5	**93.9**	**56.9**	84.2	88.1	58.0*	7.8	45.5	61.5	55.0*	37.5*	54.6*	64.3*
REPLACEAPI	98.0	13.3*	61.4*	74.4*	78.8*	9.0*	50.7*	64.1*	**97.6***	**14.2***	**61.8***	**75.1***	**86.3***	**78.5***	**85.9***	**88.5**
AdaLoRA-L	**98.4**	**89.7**	**95.1**	**96.9**	93.0	55.6	**85.1**	**88.6**	75.3	10.3	52.6	67.0	80.0	69.1	79.5	84.4

confirm that these improvements are significant. This result clearly demonstrates that our proposed layer localization method not only substantially enhances the Specificity of AdaLoRA but also preserves its performance across the other three dimensions.

> **Answer to RQ3:** AdaLoRA-L, leveraging the localization of Specific API Layers and Common API Layers, accurately identifies the storage locations of knowledge for different APIs, enabling targeted editing of API knowledge. This approach not only effectively improves Specificity but also maximally retains performance in Effectiveness, Generalization, and Portability.

4.4 RQ4: Is AdaLoRA-L more effective and efficient than REPLACEAPI?

We further compare AdaLoRA-L with `REPLACEAPI` [57], a post-hoc approach that detects deprecated APIs in model outputs, removes the deprecated API and subsequent tokens, appends the corresponding up-to-date API, and feeds the modified prefix back to the LLM for regeneration. In terms of Effectiveness (AEM), AdaLoRA-L and `REPLACEAPI` achieve comparable performance. Although `REPLACEAPI` explicitly uses token-level replacement, it does not always reach 100% AEM because the LLM may further modify the replaced API during subsequent regeneration. For example, after substituting the deprecated `torch.eig` with `torch.linalg.eig`, the model may alter it to `torch.linalg.eigvalsh`. AdaLoRA-L achieves higher Generalization (AEM) than `REPLACEAPI`. This is because a subset of generalization samples does not initially trigger deprecated APIs (as discussed in Section 3.2); consequently, `REPLACEAPI` cannot apply replacements, whereas AdaLoRA-L, having been edited on similar contexts, directly generates the correct up-to-date APIs. By contrast, `REPLACEAPI` achieves higher Portability (over 96% AEM), while AdaLoRA-L attains 74.3%–84.3%. This gap arises because model editing is optimized on effectiveness data and may not fully generalize to substantially different contexts, i.e., portability data. At the same time, AdaLoRA-L yields consistently higher EM scores than REPLACEAPI in Effectiveness and Generalization, with particularly large gaps in Effectiveness (41.1%–76.4%). This suggests that AdaLoRA-L is more effective at generating complete and parameter-correct API usages, whereas `REPLACEAPI` mainly guides the API-name replacement and still relies on the base LLM to complete the surrounding code. Thus, the results indicate a trade-off between post-hoc API-name correction and knowledge-level model editing, rather than showing that one approach dominates the other in all dimensions. While `REPLACEAPI` achieves higher Specificity by avoiding parameter updates, AdaLoRA-L nonetheless attains consistently high Specificity, with AEM exceeding 80%. To further assess whether editing

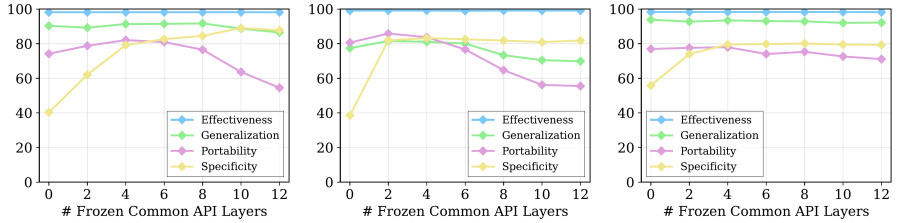


Fig. 7. The impact of the number of frozen Common API Layers on AdaLoRA-L.

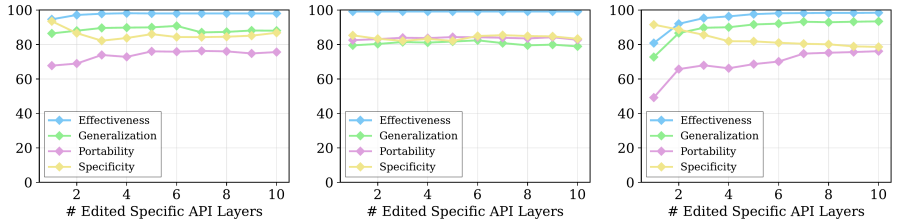


Fig. 8. The impact of the number of edited Specific API Layers on AdaLoRA-L.

affects general coding ability, we evaluate the edited models on HumanEval [7]. After AdaLoRA-L editing, pass@1 scores remain largely unchanged (46.3%/31.7%/29.3% vs. 46.9%/31.1%/29.9%), whereas AdaLoRA leads to a substantial degradation (36.6%/23.7%/26.2%). In terms of efficiency, `REPLACEAPI` incurs significantly higher inference cost, as it requires two rounds of generation. On EDAPIBench, its average inference time per sample is 3.31, 1.83, and 1.87 seconds across the three models, compared to 1.56, 0.89, and 0.87 seconds for AdaLoRA-L. This additional generation also increases token consumption, especially in long-context scenarios.

> **Answer to RQ4:** AdaLoRA-L and REPLACEAPI show complementary strengths. AdaLoRA-L achieves stronger Generalization and parameter-correct API generation with lower inference cost, while REPLACEAPI provides high Portability and Specificity without modifying model parameters, at the cost of an additional regeneration step.

5 Discussion

5.1 Impact of Hyperparameters on the Performance of AdaLoRA-L

For RQ3, we set the number of frozen Common API Layers to 8 for DeepSeek-Coder and Qwen2.5-Coder. Given that CodeGemma has fewer total layers (18 compared to 24 for DeepSeek-Coder and 36 for Qwen2.5-Coder), we set this value to 4. Similarly, the number of edited Specific API Layers is set to 4 for CodeGemma and 8 for the other two models. In this subsection's experiment, we first fix the number of edited Specific API Layers at 4 for CodeGemma and 8 for the other two models, then vary the number of frozen Common API Layers within the range of 0–12. As shown in Figure 7, increasing the number of frozen Common API Layers enhances the model's Specificity. Meanwhile, all models exhibit minimal changes in Effectiveness and Generalization—this is due to the fact that Generalization here targets edited inputs that only undergo syntactic rephrasing (the core of the input related to API editing remains unchanged), and both metrics depend on API-specific knowledge stored in Specific API Layers. The general knowledge in Common API Layers has low relevance to this kind of input scenario, so it has little impact on the two metrics. In contrast, Portability decreases as the number of frozen Common API Layers increases. Portability reflects the model's ability to handle any input x_p, which represents a real-world input that is both semantically and syntactically different from the original editing input x and is completed by

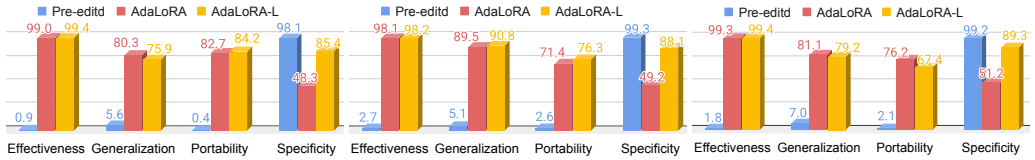


Fig. 9. The performance of AdaLoRA-L across different scales of Qwen2.5-Coder.

the original LLM f using the deprecated API y_d. Compared to the inputs for Generalization, such inputs require a greater amount of general knowledge to be processed effectively. Therefore, as more Common API Layers are frozen, the model's Portability declines.

Subsequently, with the number of frozen Common API Layers fixed, we vary the number of edited Specific API Layers from 1 to 10. As shown in Figure 8, a consistent trend is observed across all three models (although it is less pronounced for CodeGemma): increasing the number of edited Specific API Layers—up to 8 for Qwen2.5-Coder and DeepSeek-Coder, and up to 4 for CodeGemma—leads to improvements in Effectiveness, Generalization, and Portability, while Specificity decreases. This behavior can be attributed to the expanded scope of parameter updates introduced by editing more layers. On the one hand, updating a larger number of parameters increases the likelihood of interfering with knowledge unrelated to the target API, resulting in reduced Specificity. On the other hand, involving more layers in the editing process enhances the model's capacity to internalize the updated API knowledge, thereby improving editing success and facilitating better generalization to related inputs, which collectively boosts the three metrics. For CodeGemma, the gains from increasing the number of edited layers are comparatively limited. We attribute this to its relatively shallow architecture (18 layers, compared to 24 for DeepSeek-Coder and 36 for Qwen2.5-Coder), where semantic knowledge is more evenly distributed across layers. As a result, the difference in semantic capacity between single-layer and multi-layer editing is marginal, making it difficult for CodeGemma to benefit substantially from editing additional layers. Based on these observations and the need to balance performance across all evaluation dimensions, we adopt the final hyperparameter settings for the three models as described in Section 4.3.2 (RQ3).

5.2 Impact of Model Scale on the Performance of AdaLoRA-L

This section investigates whether the LLM scale influences AdaLoRA-L's performance. We select the Qwen2.5-Coder series due to its more diverse range of model sizes compared to CodeGemma and DeepSeek-Coder, enabling us to test AdaLoRA-L and AdaLoRA across three distinct scales: 1.5B, 3B, and 7B parameters. The results in Figure 9 demonstrate that AdaLoRA-L consistently achieves stable improvements in Specificity compared to AdaLoRA across all model scales, while maintaining robust performance across the remaining three dimensions regardless of model size.

5.3 Impact of Different Libraries on Model Editing Performance

This section analyzes the editing performance of AdaLoRA-L on APIs from different libraries. To ensure statistical significance, we focus on the four most sample-rich libraries: PyTorch, SciPy, scikit-learn, and TensorFlow. Figures 10, 11, and 12 present the API Exact Match (AEM) results across the four evaluation dimensions for the three models, respectively. Our observations reveal that Effectiveness and Generalization show little variation across libraries. In contrast, Portability and Specificity exhibit more noticeable differences across libraries. For example, on DeepSeek-Coder and CodeGemma, PyTorch achieves notably lower Specificity scores (68.2 and 75.3, respectively) compared to TensorFlow (91.3 and 88.6). Across all three models, SciPy consistently exhibits the lowest Portability, averaging 72.4, whereas TensorFlow achieves the highest average Portability of

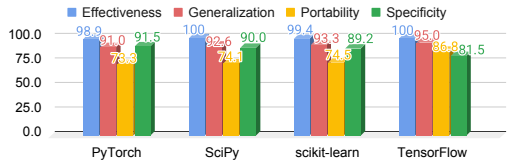


Fig. 10. The performance of AdaLoRA-L across different libraries on Qwen2.5-Coder.

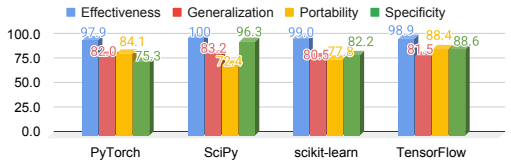


Fig. 11. The performance of AdaLoRA-L across different libraries on CodeGemma.

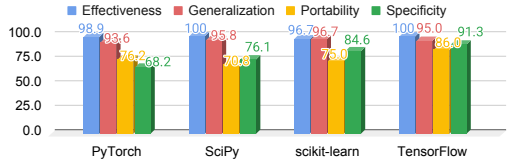


Fig. 12. The performance of AdaLoRA-L across different libraries on DeepSeek-Coder.

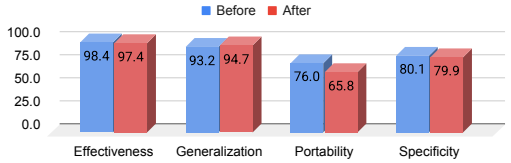


Fig. 13. The impact of API deprecation timing (Before vs. After LLM training data cutoff) on AdaLoRA-L.

87.1. Overall, while no strong or consistent patterns emerge across libraries, AdaLoRA-L demonstrates robust performance across all four evaluation dimensions for these libraries, confirming its capability to effectively edit deprecated API knowledge regardless of the library.

5.4 Impact of API Deprecation Timing on Model Editing Performance

This section investigates whether the timing of API deprecation affects the performance of deprecated API knowledge editing. We first analyze the official deprecation dates of each deprecated API in `EDAPIBench`, categorizing them as deprecated either before or after each model's training data cutoff. Statistics show all APIs are deprecated before CodeGemma and Qwen2.5-Coder's cutoffs. For DeepSeek-Coder, 6 APIs (38 editing instances) are deprecated post-cutoff, while 70 APIs (1,033 instances) are deprecated pre-cutoff. Thus, Figure 13 presents AdaLoRA-L results on DeepSeek-Coder across these two groups in terms of AEM. The results show that the deprecation timing relative to DeepSeek-Coder's cutoff has minimal impact on Effectiveness, Generalization, or Specificity. However, Portability is slightly lower for APIs deprecated post-cutoff. A plausible explanation is as follows: during DeepSeek-Coder's pre-training, the model only encountered non-deprecated usages of these 6 post-cutoff APIs, without exposure to their deprecation status or any updated alternatives. In contrast, for the 70 pre-cutoff APIs, the training data likely included both deprecated and up-to-date usages. This discrepancy causes DeepSeek-Coder to form a strongly entrenched understanding of the 6 post-cutoff APIs as valid, non-deprecated entities. Consequently, when updating the deprecated API knowledge for these 6 APIs, the newly injected information must counteract the model's rigid pre-existing understanding of "valid usage"—rendering this new knowledge less robust and ultimately resulting in slightly lower Portability.

5.5 When to Use AdaLoRA-L and REPLACEAPI

The comparison with REPLACEAPI highlights a practical trade-off between model editing and post-hoc API replacement. REPLACEAPI has several attractive properties: it is model-agnostic, does not require access to model parameters, and can be deployed whenever a deprecated-to-up-to-date API mapping is available. Since it does not modify the model, it naturally preserves unrelated model behavior, which explains its strong Specificity results. Therefore, REPLACEAPI is a suitable choice when developers mainly need to prevent deprecated API names from appearing in generated code, when the target model is closed-source or cannot be edited, or when deployment simplicity is more important than reducing per-query inference cost. AdaLoRA-L is more suitable when

the goal is to update the model's internal API knowledge rather than only rewrite the generated API name. This distinction becomes important when API evolution changes function signatures, parameter names, return values, or expected usage patterns. In these cases, replacing the API name alone may still leave the LLM responsible for generating the correct surrounding code from outdated knowledge. AdaLoRA-L is also preferable in interactive or high-throughput settings, such as IDE code completion, where repeated detection and regeneration can increase latency and token consumption. Thus, the two approaches should be viewed as serving different deployment needs. REPLACEAPI offers a non-invasive solution for API-name-level correction, especially for closed-source models or low-frequency use cases. AdaLoRA-L is better suited for open-source models and production settings that require parameter-correct API usage and lower inference overhead.

6 Threats of Validity

(1) Even with fixed temperature and greedy decoding, LLM outputs vary across runs. This causes occasional non-zero pre-edit AEM scores and Specificity AEM rarely reaching 100%. To reduce these effects, each experiment is repeated five times, and the median is reported. (2) Our study focuses on single-line API completion—checking whether the edited LLM generates the correct up-to-date API—rather than multi-line code or end-to-end functional correctness. This is because (a) multi-line correctness requires task-specific test cases, difficult to standardize across 3,000+ instances, and (b) prompts often lack enough context for the LLM to infer full multi-line logic, making evaluation prone to bias. Accordingly, following established practices in code completion studies [67, 69], we focus on single-line evaluation. (3) For AdaLoRA-L, Effectiveness and Generalization EM are high, but Portability EM is comparatively lower, despite relatively strong BLEU and ROUGE-L scores. This may be partly because the LLM generates syntactically valid API calls with arguments that do not match the ground truth—a limitation also observed in existing single-line code-completion methods, where EM rarely exceeds 0.3 for LLMs with up to 3B parameters [72]. Another contributing factor is that model editing is performed only on Effectiveness data, leaving the model undertrained on Portability scenarios. Enhancing Portability EM is a promising direction for future work, for example by providing updated API descriptions or documentation during editing to improve the model's ability to generalize API usage. (4) Our benchmark currently relies on deprecated-to-up-to-date API mappings from Wang et al. [57]. Extending it to additional new APIs would require manually collecting new mappings.

7 Related Works

API Evolution. Prior studies [39, 46, 47, 49] have examined the motivations behind API deprecation and how developers respond to the changes. Common reasons for deprecating APIs include improving code readability, reducing redundancy, addressing poor coding practices, and fixing bugs. Deprecated APIs can impact hundreds of dependent projects [45], particularly as developers struggle to keep pace with fast-evolving software ecosystems [30, 62]. For instance, McDonnell et al. [35] found that only 22% of deprecated API usages are eventually migrated to replacement APIs. Hora et al. [17] observed that while developers invest considerable effort in identifying and adopting alternatives, most projects fail to take timely action to address deprecations. Such delays lead to the silent accumulation of technical debt, complicating future migrations as multiple API changes must be resolved simultaneously [48]. Against this backdrop, deprecated APIs pose a critical problem for LLMs—a challenge amplified by their growing integration into software development workflows. Wang et al. [57] found that 37.4% of GPT-3.5's API predictions are deprecated, motivating our work to update LLMs' API knowledge via model editing to keep pace with evolving APIs.

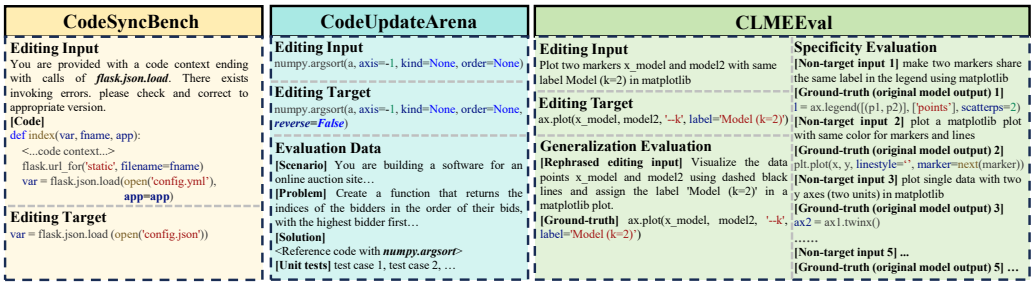


Fig. 14. The three benchmarks similar to our EDAPIBench.

LLM API Knowledge Updating. In the broader domain of LLM API knowledge updating, three closely related studies are Wang et al.'s CodeSyncBench [56], Liu et al.'s CodeUpdateArena [32], and Chhetri et al.'s empirical study [8]. The first two focus on evaluating LLMs' ability to adapt to API parameter changes (e.g., addition or removal of parameters) and generate code with correct parameter usage. CodeSyncBench focuses on API parameter correction tasks, such as fixing code like `var= flask.json.load(open('config.yml'), app=app)` to the correct form `var = flask.json.load(open('config.json'))` (Figure 14) to test whether LLMs can identify and fix parameter mismatches. CodeUpdateArena employs LLM-synthesized code generation tasks that do not fully reflect real-world scenarios and often rely on supposedly updated API parameters. For example, one task instructs the LLM to generate code using `numpy.argsort` with a newly introduced `reverse` parameter (Figure 14). However, `numpy.argsort` does not actually support a `reverse` parameter, indicating that such tasks are based on synthetic and unrealistic API specifications.

Regarding methods for updating API parameter knowledge, Liu et al. [32] relied solely on LoRA, which yielded limited Effectiveness: the maximum improvement in Pass@1 reached only 6%. Wu et al.'s ReCode [64] investigated rule-based reinforcement learning (RL) for API knowledge updating and reported substantial Pass@1 improvements on CodeUpdateArena. Wang et al. [56], meanwhile, employed SFT-LoRA [43] alongside three RL-based approaches: DPO [44], SimPO [38], and ORPO [16]. While these RL methods also use LoRA for fine-tuning, they differ from standard LoRA in their fine-tuning signals: RL relies on reward-driven feedback (e.g., DPO's preference rankings, ORPO's pairwise comparisons) rather than direct supervised signals for targeted knowledge updates. Wang et al. [56] found that DPO achieved the best performance, improving CodeBLEU scores from 32.68 (original model) to 44.95, though the absolute improvement remained modest. They also noted that RL methods required two to three times more training time than standard LoRA and involved additional reward engineering. These factors conflict with our focus on lightweight, efficient editing of deprecated API knowledge, where simplicity and speed are prioritized. Consequently, our study concentrates exclusively on LoRA-based techniques and does not incorporate RL-based approaches.

Another key distinction between these benchmarks and our EDAPIBench lies in the evaluation of Specificity. Both CodeSyncBench and CodeUpdateArena assess Specificity indirectly by tracking changes in Pass@1 scores on the HumanEval benchmark after fine-tuning. However, we argue that a more rigorous evaluation—aligned with EDAPIBench's design—should directly verify whether updated LLMs can correctly generate parameters for other non-updated APIs within CodeSyncBench and CodeUpdateArena. This direct assessment ensures that the knowledge update process does not inadvertently degrade the model's existing knowledge of unaffected APIs, providing a more precise and targeted measure than indirect metrics such as HumanEval Pass@1.

Meanwhile, concurrent work by Chhetri et al. [8] also focuses on addressing API deprecation using five editing methods (all covered by the ten methods we investigate). They select 797 HumanEval and MBPP tasks with API calls and create synthetic deprecation scenarios—for example, replacing

`math.sqrt()` (treated as deprecated) with `math.square_root()` (treated as an up-to-date API). Notably, these scenarios are also unrealistic, as `math.square_root()` does not exist in reality.

Model Editing. Model editing is an active NLP research area, with many techniques and benchmarks developed to refine general-purpose LLMs. For example, $\text{WikiData}_{\text{recent}}$ [9], ZsRE [26], and KnowEdit [73] are widely used benchmarks for factual knowledge editing. However, a key limitation of these datasets is that they do not verify whether models actually produce incorrect or hallucinated outputs before model editing. As a result, using these benchmarks to evaluate post-editing performance makes it difficult to accurately assess the effectiveness of different knowledge editing techniques in correcting hallucinations. In contrast, our `EDAPIBench` triggers edits only when models actually generate deprecated APIs, providing a more realistic and targeted evaluation scenario. The application of model editing to software engineering tasks remains relatively limited. Gu et al. [12] proposed MENT, which repaired next-token errors in code generation by patching specific neurons in LLMs. Liu et al. [31] developed CREME to enhance LLMs' robustness against prompt perturbations in code generation (e.g., typos) via targeted parameter updates in robustness-sensitive layers. Nevertheless, these approaches focus on code generation tasks where there is no fixed correct output. This differs from our focus on factual knowledge updating, and thus these methods are not included in our evaluation. A more closely related work is Li et al.'s `CLMEEval` [28]—a model editing benchmark derived from `CoNaLa` [65] and `CodeSearchNet` [22]. However, the task setup in CLMEEval is somewhat detached from practical scenarios. For example (as illustrated in Figure 14), given a simple plotting description (e.g., "*Plot two markers x_model and model2 with same label Model (k=2) in matplotlib*") as the editing input, the benchmark directly sets the target output to `ax.plot(x_model, model2, '--k', label='Model (k=2)')` without first verifying whether the LLM already generates the correct code from the prompt prior to editing. This diverges from a realistic editing paradigm, where edits are applied only when the model produces errors. Furthermore, CLMEEval relies on exact-match evaluation, which is limiting for code generation tasks since LLMs may generate functionally correct code that differs syntactically from the target. The benchmark also evaluates generalization solely through simple input paraphrases (e.g., textual rewrites of the original task), with limited diversity in paraphrase types. In contrast, our benchmark triggers editing only when LLMs output deprecated APIs. We introduce an "API Exact Match" metric to more precisely measure editing Effectiveness. To assess Generalization, we utilize GPT-4.1-generated code rewrites and incorporate a Portability dimension that evaluates whether the edited model can correctly complete the updated API calls across different editing instances, where the original model also completes the prompt with the same deprecated API.

8 Conclusion and Future Works

This study presents `EDAPIBench`, a mostly automated benchmark for editing deprecated API knowledge in LLMs, covering over 70 deprecated APIs from 8 Python libraries with 900+ instances per model. Unlike most existing benchmarks in NLP and software engineering, it only includes cases where LLMs initially generate deprecated APIs, ensuring meaningful needs-editing evaluations. Thus, `EDAPIBench` provides a standardized, rigorous platform for model editing research, benefiting both the software engineering and broader NLP communities. We evaluate 10 editing methods, finding AdaLoRA most effective but limited in Specificity, motivating our AdaLoRA-L variant. Overall, AdaLoRA-L offers a lightweight approach for updating deprecated API knowledge, enabling LLMs to generate more reliable and up-to-date code without disrupting other API behaviors. Future work will expand `EDAPIBench` to cover more deprecated APIs across additional programming languages.

Data Availability

Our `EDAPIBench`, the code for its construction, and model editing source code are available in [3].

Acknowledgments

This research was supported by National Natural Science Foundation of China under Grant No. 62502440 and Zhejiang Provincial Natural Science Foundation of China under Grant No. LQN26F020003.

References

[1] [n. d.]. *API Lifecycle Stages*. https://developers.meetmarigold.com/engage/terms/versioning-deprecation/#api-lifecycle-stages
[2] [n. d.]. *GPT-4.1*. https://openai.com/index/gpt-4-1/
[3] [n. d.]. *Our source code*. https://github.com/GuanchengLin/EDAPIBench
[4] [n. d.]. *Pytorch: A python package that provides tensor computation and deep neural networks*. https://pytorch.org/
[5] [n. d.]. *Sourcegraph*. https://sourcegraph.com/search
[6] Aakash Bansal, Sakib Haque, and Collin McMillan. 2021. Project-level encoding for neural source code summarization of subroutines. In *2021 IEEE/ACM 29th International Conference on Program Comprehension (ICPC)*. IEEE, 253–264. doi:10.1109/ICPC52881.2021.00032
[7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021). doi:10.48550/arXiv.2107.03374
[8] Vinaik Chhetri, Moghis Fereidouni, A. B Siddique, and Umar Farooq. 2026. Understanding Robustness of Model Editing in Code LLMs. (2026). arXiv:2511.03182 [cs.SE] doi:10.48550/arXiv.2511.03182
[9] Roi Cohen, Eden Biran, Ori Yoran, Amir Globerson, and Mor Geva. 2024. Evaluating the ripple effects of knowledge editing in language models. *Transactions of the Association for Computational Linguistics* 12 (2024), 283–298. doi:10.1162/tacl_a_00644
[10] Junfeng Fang, Houcheng Jiang, Kun Wang, Yunshan Ma, Jie Shi, Xiang Wang, Xiangnan He, and Tat-Seng Chua. 2025. Alphaedit: Null-space constrained knowledge editing for language models. In *International Conference on Learning Representations*, Vol. 2025. 16366–16396. doi:10.48550/arXiv.2410.02355
[11] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. Codebert: A pre-trained model for programming and natural languages. In *Findings of the association for computational linguistics: EMNLP 2020*. 1536–1547. doi:10.18653/v1/2020.findings-emnlp.139
[12] Jian Gu, Aldeida Aleti, Chunyang Chen, and Hongyu Zhang. 2023. Neuron Patching: Semantic-based Neuron-level Language Model Repair for Code Generation. *ACM Transactions on Software Engineering and Methodology* (2023). doi:10.1145/3799426
[13] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yifan Wu, YK Li, et al. 2024. DeepSeek-Coder: when the large language model meets programming–the rise of code intelligence. *arXiv preprint arXiv:2401.14196* (2024). doi:10.48550/arXiv.2401.14196
[14] Qi Guo, Xiaohong Li, Xiaofei Xie, Shangqing Liu, Ze Tang, Ruitao Feng, Junjie Wang, Jidong Ge, and Lei Bu. 2024. Ft2ra: A fine-tuning-inspired approach to retrieval-augmented code completion. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 313–324. doi:10.1145/3650212.3652130
[15] Tom Hartvigsen, Swami Sankaranarayanan, Hamid Palangi, Yoon Kim, and Marzyeh Ghassemi. 2023. Aging with grace: Lifelong model editing with discrete key-value adaptors. *Advances in Neural Information Processing Systems* 36 (2023), 47934–47959. doi:10.52202/075280-2079
[16] Jiwoo Hong, Noah Lee, and James Thorne. 2024. Orpo: Monolithic preference optimization without reference model. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 11170–11189. doi:10.18653/v1/2024.emnlp-main.626
[17] André Hora, Romain Robbes, Nicolas Anquetil, Anne Etien, Stéphane Ducasse, and Marco Tulio Valente. 2015. How do developers react to api evolution? the pharo ecosystem case. In *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 251–260. doi:10.1109/ICSM.2015.7332471
[18] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685* (2021). doi:10.48550/arXiv.2106.09685
[19] Mingzhe Hu and Yu Zhang. 2023. An empirical study of the Python/C API on evolution and bug patterns. *Journal of Software: Evolution and Process* 35, 2 (2023), e2507. doi:10.1002/smr.2507
[20] Baixiang Huang, Canyu Chen, Xiongxiao Xu, Ali Payani, and Kai Shu. 2025. Can knowledge editing really correct hallucinations?. In *International Conference on Learning Representations*, Vol. 2025. 88116–88149.
[21] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024). doi:10.48550/arXiv.2409.12186

[22] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. Codesearchnet challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436* (2019). doi:10.48550/arXiv.1909.09436

[23] Maliheh Izadi, Jonathan Katzy, Tim Van Dam, Marc Otten, Razvan Mihai Popescu, and Arie Van Deursen. 2024. Language models for code completion: A practical evaluation. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13. doi:10.1145/3597503.3639138

[24] Zhenlan Ji, Pingchuan Ma, Zongjie Li, Zhaoyu Wang, and Shuai Wang. 2025. Causality-Aided Evaluation and Explanation of Large Language Model-Based Code Generation. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 1374–1397. doi:10.1145/3728938

[25] Raula Gaikovina Kula, Ali Ouni, Daniel M German, and Katsuro Inoue. 2018. An empirical study on the impact of refactoring activities on evolving client-used apis. *Information and Software Technology* 93 (2018), 186–199. doi:10.1016/j.infsof.2017.09.007

[26] Omer Levy, Minjoon Seo, Eunsol Choi, and Luke Zettlemoyer. 2017. Zero-shot relation extraction via reading comprehension. In *Proceedings of the 21st Conference on Computational Natural Language Learning (CoNLL 2017)*. 333–342. doi:10.18653/v1/K17-1034

[27] Xiaopeng Li, Shasha Li, Shezheng Song, Jing Yang, Jun Ma, and Jie Yu. 2024. Pmet: Precise model editing in a transformer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 18564–18572. doi:10.1609/aaai.v38i17.29818

[28] Xiaopeng Li, Shangwen Wang, Shasha Li, Jun Ma, Jie Yu, Xiaodong Liu, Jing Wang, Bin Ji, and Weimin Zhang. 2025. Model editing for llms4code: How far are we?. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 937–949. doi:10.1109/ICSE55347.2025.00049

[29] Chin-Yew Lin. 2004. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*. 74–81.

[30] Mario Linares-Vásquez, Gabriele Bavota, Carlos Bernal-Cárdenas, Massimiliano Di Penta, Rocco Oliveto, and Denys Poshyvanyk. 2013. Api change and fault proneness: A threat to the success of android apps. In *Proceedings of the 2013 9th joint meeting on foundations of software engineering*. 477–487. doi:10.1145/2491411.2491428

[31] Shuhan Liu, Xing Hu, Kerui Huang, Xiaohu Yang, David Lo, and Xin Xia. 2025. Improving Code LLM Robustness to Prompt Perturbations via Layer-Aware Model Editing. *arXiv e-prints* (2025), arXiv–2507. doi:10.48550/arXiv.2507.16407

[32] Zeyu Leo Liu, Shrey Pandit, Xi Ye, Eunsol Choi, and Greg Durrett. 2024. Codeupdatearena: Benchmarking knowledge editing on api updates. *arXiv preprint arXiv:2407.06249* (2024). doi:10.48550/arXiv.2407.06249

[33] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. 2024. Starcoder 2 and the stack v2: The next generation. *arXiv preprint arXiv:2402.19173* (2024). doi:10.48550/arXiv.2402.19173

[34] Xiaoxue Ma, Jacky Wai Keung, Xiao Yu, Huiqi Zou, Jingyu Zhang, and Yishu Li. 2023. AttSum: A deep attention-based summarization model for bug report title generation. *IEEE Transactions on Reliability* 72, 4 (2023), 1663–1677. doi:10.1109/TR.2023.3236404

[35] Tyler McDonnell, Baishakhi Ray, and Miryung Kim. 2013. An empirical study of api stability and adoption in the android ecosystem. In *2013 IEEE International Conference on Software Maintenance*. IEEE, 70–79. doi:10.1109/ICSM.2013.18

[36] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. 2022. Locating and editing factual associations in gpt. *Advances in neural information processing systems* 35 (2022), 17359–17372. doi:10.52202/068431-1262

[37] Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. 2022. Mass-editing memory in a transformer. *arXiv preprint arXiv:2210.07229* (2022). doi:10.48550/arXiv.2210.07229

[38] Yu Meng, Mengzhou Xia, and Danqi Chen. 2024. Simpo: Simple preference optimization with a reference-free reward. *Advances in Neural Information Processing Systems* 37 (2024), 124198–124235. doi:10.52202/079017-3946

[39] Ariana Mirian, Nikunj Bhagat, Caitlin Sadowski, Adrienne Porter Felt, Stefan Savage, and Geoffrey M Voelker. 2019. Web feature deprecation: a case study for chrome. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 302–311. doi:10.1109/ICSE-SEIP.2019.00044

[40] Pavlo Molchanov, Arun Mallya, Stephen Tyree, Iuri Frosio, and Jan Kautz. 2019. Importance estimation for neural network pruning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 11264–11272. doi:10.1109/CVPR.2019.01152

[41] Zhenyu Pan, Rongyu Cao, Yongchang Cao, Yingwei Ma, Binhua Li, Fei Huang, Han Liu, and Yongbin Li. 2024. CodevBench: How Do LLMs Understand Developer-Centric Code Completion? *arXiv preprint arXiv:2410.01353* (2024). doi:10.48550/arXiv.2410.01353

[42] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*. 311–318. doi:10.3115/1073083.1073135

[43] Baolin Peng, Chunyuan Li, Pengcheng He, Michel Galley, and Jianfeng Gao. 2023. Instruction tuning with gpt-4. *arXiv preprint arXiv:2304.03277* (2023). doi:10.48550/arXiv.2304.03277

[44] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems* 36 (2023), 53728–53741. doi:10.52202/075280-2338

[45] Romain Robbes, Mircea Lungu, and David Röthlisberger. 2012. How do developers react to API deprecation? The case of a Smalltalk ecosystem. In *Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering*. 1–11. doi:10.1145/2393596.2393662

[46] Anand Ashok Sawant, Mauricio Aniche, Arie van Deursen, and Alberto Bacchelli. 2018. Understanding developers' needs on deprecation as a language feature. In *Proceedings of the 40th international conference on software engineering*. 561–571. doi:10.1145/3180155.3180170

[47] Anand Ashok Sawant, Guangzhe Huang, Gabriel Vilen, Stefan Stojkovski, and Alberto Bacchelli. 2018. Why are features deprecated? an investigation into the motivation behind deprecation. In *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. Ieee, 13–24. doi:10.1109/ICSME.2018.00011

[48] Anand Ashok Sawant, Romain Robbes, and Alberto Bacchelli. 2016. On the reaction to deprecation of 25,357 clients of 4+ 1 popular Java APIs. In *2016 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 400–410. doi:10.1109/ICSME.2016.64

[49] Anand Ashok Sawant, Romain Robbes, and Alberto Bacchelli. 2019. To react, or not to react: Patterns of reaction to API deprecation. *Empirical Software Engineering* 24, 6 (2019), 3824–3870. doi:10.1007/s10664-019-09713-w

[50] Shaznin Sultana, Sadia Afreen, and Nasir U Eisty. 2024. Code vulnerability detection: A comparative analysis of emerging large language models. *arXiv preprint arXiv:2409.10490* (2024). doi:10.48550/arXiv.2409.10490

[51] Zhensu Sun, Xiaoning Du, Zhou Yang, Li Li, and David Lo. 2024. Ai coders are among us: Rethinking programming language grammar towards efficient code generation. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1124–1136. doi:10.1145/3650212.3680347

[52] Chenmien Tan, Ge Zhang, and Jie Fu. 2024. Massive editing for large language models via meta learning. In *International Conference on Learning Representations*, Vol. 2024. 54676–54690.

[53] CodeGemma Team, Heri Zhao, Jeffrey Hui, Joshua Howland, Nam Nguyen, Siqi Zuo, Andrea Hu, Christopher A Choquette-Choo, Jingyue Shen, Joe Kelley, et al. 2024. Codegemma: Open code models based on gemma. *arXiv preprint arXiv:2406.11409* (2024). doi:10.48550/arXiv.2406.11409

[54] David Thissen, Lynne Steinberg, and Daniel Kuang. 2002. Quick and easy implementation of the Benjamini-Hochberg procedure for controlling the false positive rate in multiple comparisons. *Journal of educational and behavioral statistics* 27, 1 (2002), 77–83. doi:10.3102/10769986027001077

[55] Yao Wan, Zhou Zhao, Min Yang, Guandong Xu, Haochao Ying, Jian Wu, and Philip S Yu. 2018. Improving automatic source code summarization via deep reinforcement learning. In *Proceedings of the 33rd ACM/IEEE international conference on automated software engineering*. 397–407. doi:10.1145/3238147.3238206

[56] Chenlong Wang, Zhaoyang Chu, Zhengxiang Cheng, Xuyi Yang, Kaiyue Qiu, Yao Wan, Zhou Zhao, Xuanhua Shi, and Dongping Chen. 2025. Codesync: Synchronizing large language models with dynamic code evolution at scale. *arXiv preprint arXiv:2502.16645* (2025). doi:10.48550/arXiv.2502.16645

[57] Chong Wang, Kaifeng Huang, Jian Zhang, Yebo Feng, Lyuye Zhang, Yang Liu, and Xin Peng. 2025. Llms meet library evolution: Evaluating deprecated api usage in llm-based code completion. In *2025 ieee/acm 47th international conference on software engineering (icse)*. IEEE, 885–897. doi:10.1109/ICSE55347.2025.00245

[58] Chaozheng Wang, Zezhou Yang, Shuzheng Gao, Cuiyun Gao, Ting Peng, Hailiang Huang, Yuetang Deng, and Michael Lyu. 2025. Rag or fine-tuning? a comparative study on lcms-based code completion in industry. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 93–104. doi:10.1145/3696630.3728535

[59] Jiawei Wang, Li Li, Kui Liu, and Haipeng Cai. 2020. Exploring how deprecated python library apis are (not) handled. In *Proceedings of the 28th acm joint meeting on european software engineering conference and symposium on the foundations of software engineering*. 233–244. doi:10.1145/3368089.3409735

[60] Peng Wang, Ningyu Zhang, Bozhong Tian, Zekun Xi, Yunzhi Yao, Ziwen Xu, Mengru Wang, Shengyu Mao, Xiaohan Wang, Siyuan Cheng, et al. 2024. Easyedit: An easy-to-use knowledge editing framework for large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*. 82–93. doi:10.18653/v1/2024.acl-demos.9

[61] Song Wang, Yaochen Zhu, Haochen Liu, Zaiyi Zheng, Chen Chen, and Jundong Li. 2024. Knowledge editing for large language models: A survey. *Comput. Surveys* 57, 3 (2024), 1–37. doi:10.1145/3698590

[62] Ying Wang, Bihuan Chen, Kaifeng Huang, Bowen Shi, Congying Xu, Xin Peng, Yijian Wu, and Yang Liu. 2020. An empirical study of usages, updates and risks of third-party libraries in java projects. In *2020 IEEE International conference on software maintenance and evolution (ICSME)*. IEEE, 35–45. doi:10.1109/ICSME46990.2020.00014

[63] Robert F Woolson. 2007. Wilcoxon signed-rank test. *Wiley encyclopedia of clinical trials* (2007), 1–3. doi:10.1002/9780471462422.eoct979

[64] Haoze Wu, Yunzhi Yao, Wenhao Yu, and Ningyu Zhang. 2026. Recode: Updating code api knowledge with reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 33908–33916. doi:10.1609/aaai.v40i40.40683

[65] Pengcheng Yin, Bowen Deng, Edgar Chen, Bogdan Vasilescu, and Graham Neubig. 2018. Learning to mine aligned code and natural language pairs from stack overflow. In *Proceedings of the 15th international conference on mining software repositories*. 476–486. doi:10.1145/3196398.3196408

[66] Shiwen Yu, Ting Wang, and Ji Wang. 2022. Data augmentation by program transformation. *Journal of Systems and Software* 190 (2022), 111304. doi:10.1016/j.jss.2022.111304

[67] Xiao Yu, Lei Liu, Xing Hu, Jacky Wai Keung, Jin Liu, and Xin Xia. 2024. Fight fire with fire: How much can we trust chatgpt on source code-related tasks? *IEEE Transactions on Software Engineering* 50, 12 (2024), 3435–3453. doi:10.1109/TSE.2024.3492204

[68] Xian Zhan, Tianming Liu, Lingling Fan, Li Li, Sen Chen, Xiapu Luo, and Yang Liu. 2021. Research on third-party libraries in android apps: A taxonomy and systematic literature review. *IEEE Transactions on Software Engineering* 48, 10 (2021), 4181–4213. doi:10.1109/TSE.2021.3114381

[69] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. Repocoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2471–2484. doi:10.18653/v1/2023.emnlp-main.151

[70] Fengji Zhang, Jin Liu, Yao Wan, Xiao Yu, Xiao Liu, and Jacky Keung. 2023. Diverse title generation for Stack Overflow posts with multiple-sampling-enhanced transformer. *Journal of Systems and Software* 200 (2023), 111672. doi:10.1016/j.jss.2023.111672

[71] Fengji Zhang, Xiao Yu, Jacky Keung, Fuyang Li, Zhiwen Xie, Zhen Yang, Caoyuan Ma, and Zhimin Zhang. 2022. Improving Stack Overflow question title generation with copying enhanced CodeBERT model and bi-modal information. *Information and Software Technology* 148 (2022), 106922. doi:10.1016/j.infsof.2022.106922

[72] Lei Zhang, Yunshui Li, Jiaming Li, Xiaobo Xia, Jiaxi Yang, Run Luo, Minzheng Wang, Longze Chen, Junhao Liu, Qiang Qu, et al. 2025. Hierarchical context pruning: Optimizing real-world code completion with repository-level pretrained code llms. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 25886–25894. doi:10.1609/aaai.v39i24.34782

[73] Ningyu Zhang, Yunzhi Yao, Bozhong Tian, Peng Wang, Shumin Deng, Mengru Wang, Zekun Xi, Shengyu Mao, Jintian Zhang, Yuansheng Ni, et al. 2024. A comprehensive study of knowledge editing for large language models. *arXiv preprint arXiv:2401.01286* (2024). doi:10.48550/arXiv.2401.01286

[74] Qingru Zhang, Minshuo Chen, Alexander Bukharin, Nikos Karampatziakis, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. 2023. Adalora: Adaptive budget allocation for parameter-efficient fine-tuning. *arXiv preprint arXiv:2303.10512* (2023). doi:10.48550/arXiv.2303.10512

[75] Qingru Zhang, Simiao Zuo, Chen Liang, Alexander Bukharin, Pengcheng He, Weizhu Chen, and Tuo Zhao. 2022. Platon: Pruning large transformer models with upper confidence bound of weight importance. In *International conference on machine learning*. PMLR, 26809–26823.

[76] Zexuan Zhong, Zhengxuan Wu, Christopher D Manning, Christopher Potts, and Danqi Chen. 2023. Mquake: Assessing knowledge editing in language models via multi-hop questions. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 15686–15702. doi:10.18653/v1/2023.emnlp-main.971

[77] Chen Zhu, Ankit Singh Rawat, Manzil Zaheer, Srinadh Bhojanapalli, Daliang Li, Felix Yu, and Sanjiv Kumar. 2020. Modifying memories in transformer models. *arXiv preprint arXiv:2012.00363* (2020). doi:10.48550/arXiv.2012.00363

Received 2026-01-30; accepted 2026-04-16